

P0018r00 : Lambda Capture of ***this** by Value

Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Christian Trott
Contact: ctrrott@sandia.gov
Author: Hal Finkel
Contact: hfinkel@anl.gov
Author: Jim Reus
Contact: reus1@llnl.gov
Author: Robin Maffeo
Contact: robin.maffeo@amd.com
Author: Ben Sander
Contact: ben.sander@amd.com
Number: P0018
Version: 00
Date: 2015-09-23
URL: <https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0018.rst>
WG21: Evolution Working Group (EWG)
WG21: Concurrency and Parallelism Study Group (SG1)

1 Issue

Lambda expressions declared within a non-static member function explicitly or implicitly captures **this** to access to member variables of **this**. Both capture-by-reference [&] and capture-by-value [=] implicitly capture the **this** pointer, therefore member variables are always accessed by reference via **this**. Thus the capture-default has no effect on the capture of **this**.

```
struct S {  
    int x ;  
    void f() {  
        // The following lambda captures are currently identical  
        auto a = [&]() { x = 42 ; } // OK: transformed to (*this).x  
        auto b = [=]() { x = 43 ; } // OK: transformed to (*this).x  
        a();  
        assert( x == 42 );  
        b();  
        assert( x == 43 );  
    }  
};
```

2 Motivations for lambda capture of ***this** by Value

Truly capturing ***this** by value allows an implicitly declared closure to be copied before invoking the closure's function.

2.1 Asynchronous dispatch of lambda

Asynchronous dispatch of closures a cornerstone of parallelism and concurrency.

When a lambda is asynchronously dispatched from within a non-static member function, via **std::async** or other concurrency / parallelism dispatch mechanism, the enclosing ***this** class *cannot* be captured by value. Thus when the **future** (or other handle) to the dispatched lambda outlives the originating class the lambda's captured **this** pointer is invalid.

```
class Work {  
private:  
    int value ;  
public:  
    Work() : value(42) {}  
    std::future<int> spawn()  
    { return std::async( [=]()->int{ return value ; }); }  
};
```

```
std::future<int> foo()
{
    Work tmp ;
    return tmp.spawn();
    // The closure associated with the returned future
    // has an implicit this pointer that is invalid.
}

int main()
{
    std::future<int> f = foo();
    f.wait();
    // The following fails due to the
    // originating class having been destroyed
    assert( 42 == f.get() );
    return 0 ;
}
```

2.2 Dispatching asynchronous closures to data

Current and future hardware architectures specifically targeting parallelism and concurrency have heterogeneous memory systems. For example, NUMA regions, attached accelerator memory, and processing-in-memory (PIM) stacks. In these architectures it will often result in significantly improved performance if the closure is copied to the data upon which it operates, as opposed to moving the data to and from the closure.

For example, parallel execution of a closure on large data spanning NUMA regions will be more performant if a copy of that closure residing in the same NUMA region acts upon that data. If true a (self-contained) capture-by-value lambda closure were given to a parallel dispatch, such as in the parallelism technical specification, then the library could create copies of that closure within each NUMA region to improve data locality for the parallel computation. For another example, a closure dispatched to an attached accelerator with separate memory must be copied to the accelerator's memory before execution can occur. Thus current and future architectures *require* the capability to copy closures to data.

2.3 Onerous and error-prone work-around

A potential work-around for this deficiency is to explicitly capture a copy the originating class.

```
class Work {
private:
    int value ;
public:
    Work() : value(42) {}
    std::future<int> spawn()
    {
        return std::async( [=,tmp=*this]()->int{ return tmp.value ; });
    }
};
```

This work-around has two liabilities. First, the **this** pointer is also captured which provides a significant opportunity to erroneously reference a **this->** member instead of a **tmp.** member. Second, it is onerous and counter-productive to the introduction of asynchronously dispatched lambda expressions within existing code. Consider the case of replacing a **for** loop within a non-static member function with a *parallel for each* construct as in the parallelism technical specification.

```
class Work {
public:
    void do_something() const {
        // for ( int i = 0 ; i < N ; ++i )
        foreach( Parallel , 0 , N , [=,tmp=*this]( int i )
        {
            // A modestly long loop body where
            // every reference to a member must be modified
            // for qualification with 'tmp.'
            // Any mistaken omissions will silently fail
            // as references via 'this->'.
        }
    };
};
```

In this example every reference to a member in the pre-existing code must be modified to add the **tmp.** qualification. This onerous process must be repeated throughout an existing code base. A true lambda capture of ***this** would eliminate such an onerous and silent-error-prone process of injecting parallelism and concurrency into an large, existing code base.

2.4 Safety and productivity in parallelism and concurrency

As currently specified integration of lambda and concurrency capabilities is perilous, as demonstrated by the previous **Work** example. A lambda generated within a non-static member function *cannot* be a true (self-contained) closure and therefore cannot reliably be used with an asynchronous dispatch.

Lambda capability is a significant boon to productivity, especially when parallel or concurrent closures can be defined with lambdas as opposed to manually generated functors. If the capability to capture ***this** by value is not enabled then the productivity benefits of lambdas cannot be fully realized in the parallelism and concurrency domain.

3 Semantics of Lambda Capture of ***this** by value

Lambda captures of ***this** by value within a non-static member function is as if:

- the implicitly generated closure object type is derived from the type of ***this**,
- the closure object type is declared a **friend** of the type of ***this**,
- the closure object type has **using** statements for all members of the type of ***this** that are accessed within the lambda expression, and
- the ***this** object is copy constructed into the closure object.

Requires: The type of ***this** to be copy constructable.

Requires: Lambda capture of ***this** by value does not occur within a copy constructor, or function invoked by a copy constructor, as this would result in an infinite recursion of the copy constructor. This requirement would also be applicable to the onerous work-around.

4 Resolution Option #1: Correct Lambda Capture-by-value [=]

The semantically consistent resolution is for the *capture-default* [=] to capture ***this** by value for lambda expressions within a non-static member function. The *capture-default* [&] within a non-static member function conforms to the current capture specification for **this**.

```
struct S {
    int x ;
    void f() {
        auto a = [&]() { x = 42 ; } // OK: transformed to (*this).x
        auto b = [=]() mutable { x = 42 ; } // Modifying copy of x

        auto c = [=]() { x = 42 ; } // Error: captured copy of '*this'
                                   // and lambda function is 'const'
    }
};
```

This resolution would correct lambda capture semantics; however, it is likely to break existing code. As such we propose the following solution.

5 Resolution Option #2: Add True Lambda Capture-by-value [*]

Given that the semantically consistent resolution would break current standard behavior, a new capture mechanism is necessary to provide semantically consistent capture-by-value semantics for lambda expressions within non-static member functions.

Extend the *capture-default* and *simple-capture* to include:

capture-default:

&
=
*

simple-capture:

identifier
& *identifier*
this
***this**

The *simple-capture* ***this** declares that ***this** is to be captured by value. The *capture-default* **[*]** declares that the default capture is by value, including ***this** if the lambda expression appears within a non-static member function. Outside of a non-static member function the *capture-default* **[*]** is identical to the *capture-default* **[=]**.

5.1 Nested lambda capture

A new capture mechanism introduces a introduces new capture interaction. For non-***this** captures the interactions remain unchanged. When ***this** is captured by value via **[*]** nested captures of **this** refer to the enclosing copy of ***this**.

```
void Work::foo()
{
    auto x = [*]() { // *this is captured by value
        auto y = [&]() {
            // refer to the copy of Work contained in 'x'
            // does not refer to the original enclosing 'this'
            this->value
        };
        auto z = [=]() {
            // refer to the copy of Work contained in 'x'
            // does not refer to the original enclosing 'this'
            this->value
        };
    };
};
```

5.2 Updated example

With true lambda capture-by-value the earlier example can have the correct behavior by generating a complete closure.

```
class Work {
private:
    int value ;
public:
    Work() : value(42) {}

    std::future<int> spawn()
        // Capture-by-value is correct and the asynchronously
        // dispatched closure may outlive the originating class,
        // and may be freely copied without losing correctness.
        { return std::async( [*]()->int{ return value ; }); }

    // Trivial change to replace 'for' with 'parallel for'
    void do_something() const {
        // for ( int i = 0 ; i < N ; ++i )
        foreach( Parallel , 0 , N , [*]( int i )
        {
            // A modestly long loop body where
            // every reference to a member can be
            // safely referenced without modification.
        }
    );
};
```